

Artea Case Analysis

Team 12: Carson Pimental, Jaya Sruthi Raj Perikala, Angie Pang, Andrea Pan

1. Company/Case Breakdown

Artea is an online retailer that specializes in handmade clothing and accessories. Despite healthy engagement metrics (high time spent on the website, healthy referral rates, etc) the percentage of website traffic that has never made a transaction is alarmingly high (87%). This gap between engagement and transactions led CEO Alex Campbel and the data science team to explore strategies to encourage customers to convert. Since many visitors provide their email addresses to receive newsletters, Artea already uses personalized email marketing based on customers' browsing behavior and stated interests, which has improved email open and click-through rates. However, the company's main objective remains increasing purchases and revenue rather than simply driving engagement.

To address this issue, Artea considered offering targeted discounts to encourage purchases while avoiding large-scale promotions that could damage brand value or train customers to wait for discounts. The company conducted an A/B test involving 5,000 users who had visited the website in the past two months but had not purchased anything. 2502 users were randomly selected to receive a non-transferable 20% coupon valid for one month, while the remaining 2498 users served as a control group. The experiment collected data on transactions, revenue, browsing behavior, acquisition channels, and past purchase activity. The goal was to determine whether targeted coupon campaigns could increase conversions and revenue, and to guide decisions about how future marketing campaigns should be designed and targeted.

2. Why is A/B Testing required for the problem that Alex is trying to solve

A/B testing is required to evaluate the true impact of the coupon campaign. Without it, Artea cannot solve the counterfactual problem, that is, there is no way to separate customers who were genuinely persuaded by the coupon from those who would have purchased anyway, making it impossible to measure the true incremental impact of the discount. Giving discounts to the latter group only destroys margins without any incremental benefit, which is precisely Alex's concern. Simply sending coupons to all customers could be risky because discounts may reduce profit margins and encourage customers to buy only when promotions are offered. A/B testing allows

the company to causally measure the effect of the coupon campaign by comparing two randomly assigned groups of users:

- Treatment group: users who receive the 20% coupon
- Control group: users who do not receive the coupon

Because the customers are randomly assigned, any difference in transactions or revenue between the two groups can be attributed to the coupon itself rather than other factors such as browsing behavior or acquisition channel. This experimental approach helps Artea:

- Determine whether coupons actually increase purchases and revenue
- Quantify the true incremental impact of the promotion
- Decide whether coupons should be sent to all customers, only certain segments, or not used at all.

3. Did the coupon increase transactions? Did it increase revenues?

The coupon effectively increased transactions, as we see the coupon recipients had 66 more transactions than the non-recipients. The average number of transactions after the experiment for customers who received the coupon was 0.1519, while the average for customers who did not receive the coupon was 0.1257. This represents an incremental increase of 0.0262 transactions per user, or approximately a 20.8% increase in transactions, indicating that the coupon successfully encouraged more customers to make purchases.

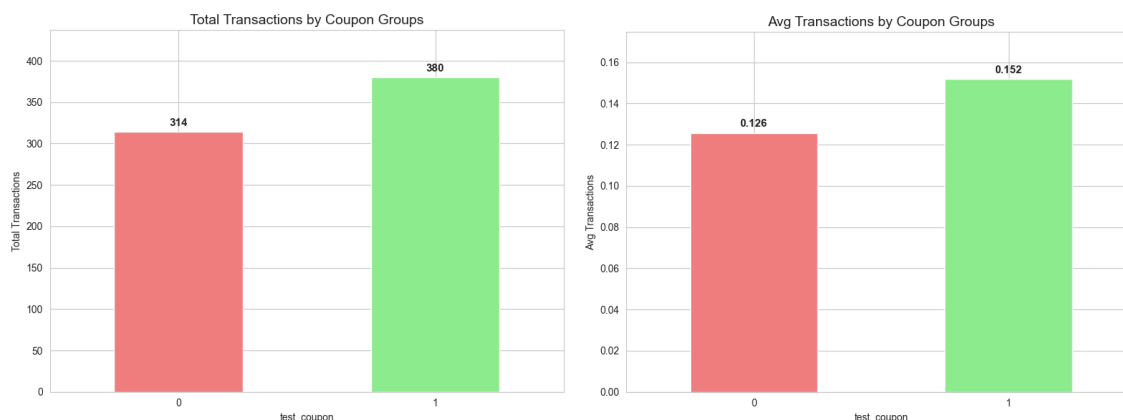


Fig 1: Impact of coupon treatment on total and average transactions in the A/B test

However, the coupon did not lead to an increase in revenue. The average revenue after the experiment for customers who received the coupon was \$7.54, while the average revenue for customers who did not receive the coupon was \$7.78. This results in an incremental change of $-\$0.24$ per user, suggesting that the 20% discount reduced the revenue generated per customer despite increasing purchase activity.

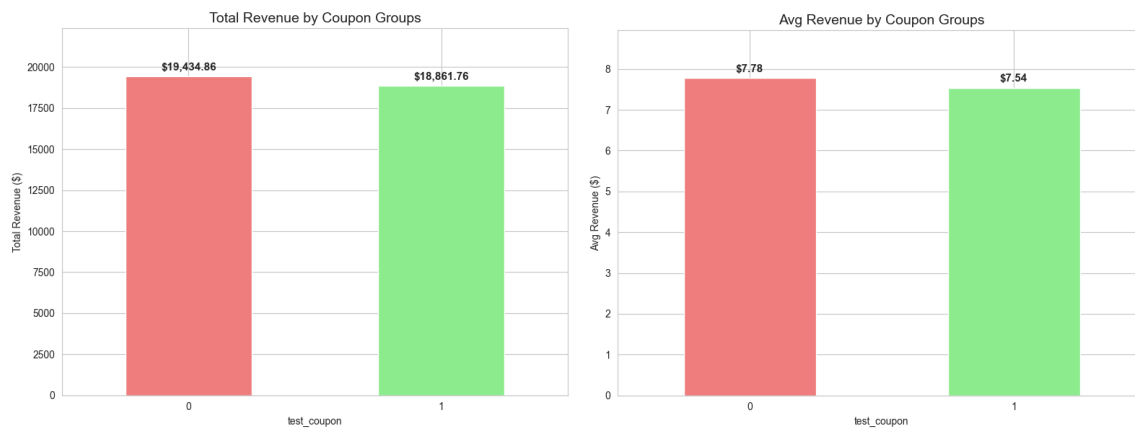


Fig 2: Impact of coupon treatment on total and average revenue in the A/B experiment

4. Which customers should be targeted in the next targeting campaign? How many transactions and how much revenues would you expect from that targeting strategy? What are the risks of this targeting strategy?

4.1. Identifying the Customers to Target

The objective of the targeting strategy is to identify customers for whom a coupon generates incremental transactions and revenue, rather than simply rewarding customers who would have purchased regardless of receiving a discount. Therefore, the focus of the targeting policy is on customers who are most responsive to the coupon treatment, as measured by the causal impact observed in the A/B experiment.

To achieve this, the experimental data was analyzed using a regression model with interaction terms between the coupon indicator and several customer attributes, including browsing behavior, past purchase activity, and acquisition channel. These interaction terms allow us to estimate how the effectiveness of the coupon varies across customer segments.

Using the model estimates, uplift was calculated for both transactions and revenue within each segment. Uplift is defined as:

- Transaction uplift = Average transactions (coupon) – Average transactions (no coupon)
- Revenue uplift = Average revenue (coupon) – Average revenue (no coupon)

Segments that demonstrate positive uplift in both transactions and revenue represent the most appropriate targets for future campaigns, as the coupon generates additional purchases without reducing overall revenue.

As shown in Figure 3, customers who added items to their shopping cart, have zero to two past purchases, and were acquired through Instagram, Facebook, or referral channels exhibit the strongest positive revenue uplift. These customers appear to be highly engaged with the website but may require a small incentive to complete their purchase. In contrast, customers with a larger number of past purchases are more likely to purchase even without discounts, meaning that sending coupons to these customers would primarily reduce margins rather than generate additional revenue.

Based on these findings, the recommended targeting rule for the next campaign is to send coupons to customers who added items to their cart, have zero to two past purchases, and were acquired through Instagram, Facebook, or referral channels, as this segment is most likely to generate positive incremental impact from the coupon campaign.

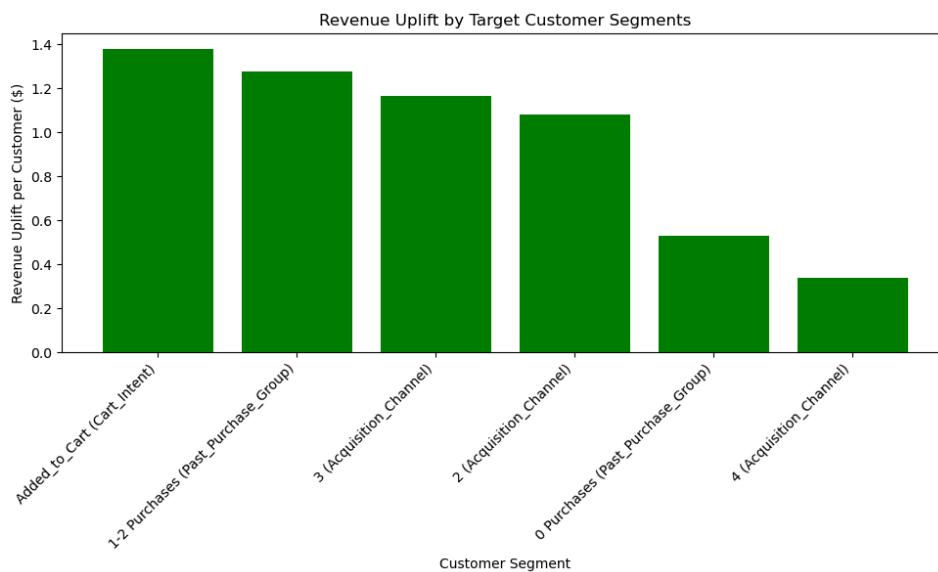


Fig 3: Revenue Uplift Across Key Customer Segments from the A/B Test

4.2. Expected Impact of the Targeting Strategy

The targeting criteria were applied to the Next_Campaign customer list to identify users who meet the conditions of having added items to their cart, having zero to two past purchases, and being acquired through Instagram, Facebook, or referral channels.

Applying this rule identifies 668 customers, representing approximately 11.13% of the available customer pool, as suitable targets for the coupon campaign.

The targeting strategy is expected to generate approximately 51 incremental transactions relative to not sending coupons. In terms of revenue, sending coupons to this targeted group is projected to generate approximately \$9,704.70 in total revenue, corresponding to an incremental revenue increase of about \$922.51 compared to not sending coupons.

These results highlight the advantage of a targeted coupon strategy. While sending coupons to all customers increased transactions but reduced overall revenue, focusing only on customers with positive revenue uplift enabled Artea to increase both transactions and revenue while minimizing unnecessary discounting.

4.3. Risks of the Targeting Strategy

Although the targeting strategy is expected to generate positive incremental revenue, several risks should be considered when implementing it in future campaigns:

- Limited Generalization of A/B Test Results

The targeting rule is based on a single experiment. Changes in customer behavior, acquisition channels, or browsing patterns over time may reduce the accuracy of the estimated uplift in future campaigns.

- Promotion Dependency

Repeated coupon campaigns may encourage customers to wait for discounts before making purchases, potentially reducing full-price conversions and eroding profit margins.

- Potential Bias in Targeting Rules

Targeting based on acquisition channels or behavioral patterns may unintentionally exclude certain customer groups from promotions. Therefore, the targeting policy should be regularly monitored to ensure fairness.

To mitigate these risks, Artea should continue running experiments and periodically update its targeting strategy based on new data and customer behavior patterns.

5. What do you learn from the demographics data? Would you change the targeting policy from the first part of the case?

The demographic analysis shows that the coupon campaign did not produce statistically significant differences in revenue across the groups examined. Instead of comparing group means directly, we tested whether coupon effectiveness differed by demographic group using interaction terms between the coupon status and demographic variables. Specifically, the interaction terms `test_coupon:minority` and `test_coupon:non_male` were included to evaluate whether the effect of receiving a coupon varied by minority status or gender. Both interaction terms were not statistically significant, with p-values of 0.639 and 0.164, respectively.

These results indicate that the coupon's impact on revenue does not meaningfully differ across demographic groups, suggesting that minority status and gender do not significantly affect coupon effectiveness. As a result, demographic attributes should not be used to modify the targeting strategy. Instead, the company should focus on behavioral indicators, such as browsing activity or shopping cart engagement, which are more directly related to purchase intent.

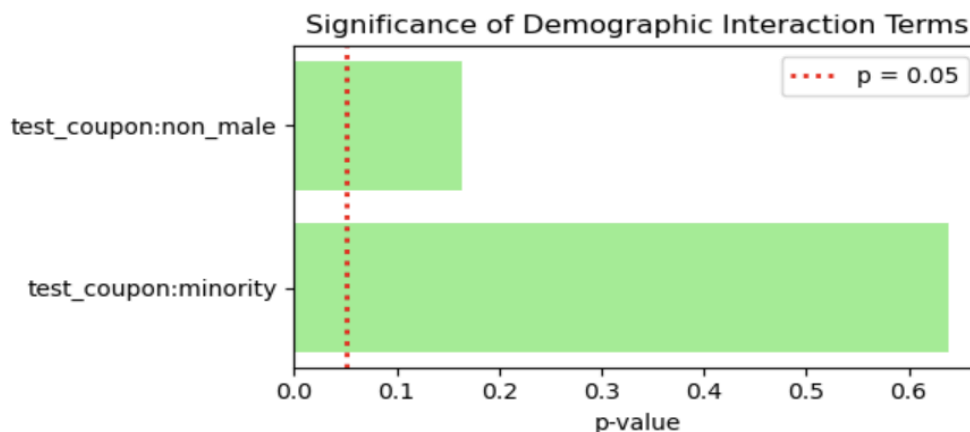


Fig 4: Significance of demographic interaction terms in the coupon response model

Previously, Facebook and Instagram showed significant uplift as acquisition channels, but they disproportionately attract non-minority customers. Google also attracts many non-minority users, but compared to Facebook and Instagram, it brings in a larger share of minority customers. Although Google did not demonstrate statistically significant uplift, it presents an opportunity to reach a more diverse customer base. If the company is willing to accept slightly lower short-term profitability, investing in Google as an acquisition channel could help expand and balance the diversity of the customer pool over time.

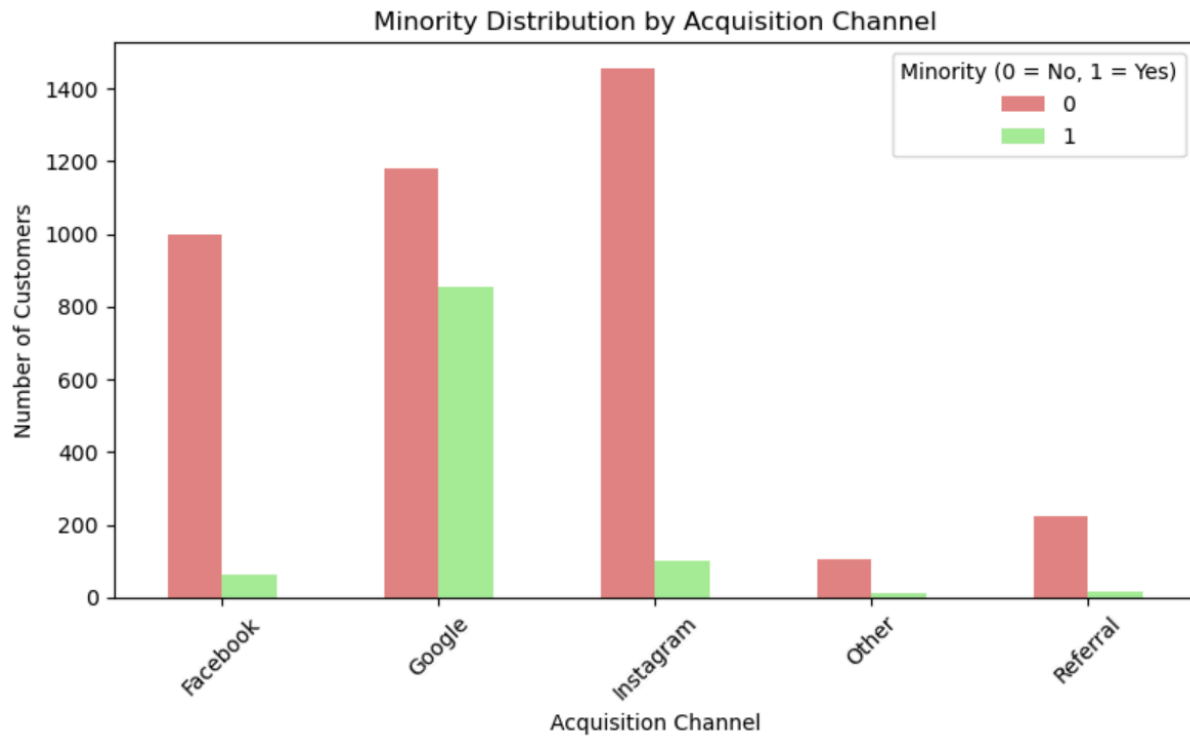


Fig 4: Distribution of minority and non-minority customers across acquisition channels
Are there any ethical concerns from collecting and using customer-level demographic data?

There are several ethical concerns in regards to collecting and utilizing customer-level demographic data. Using attributes such as gender or ethnicity/racial background for targeting can lead to discrimination or unfair treatment, despite being unintended. This concept is known as disparate impact, where a seemingly neutral strategy produces outcomes that systematically disadvantage certain groups. This in certain industries is also considered illegal and therefore must be a primary consideration when developing strategies with demographic data. Privacy is another major concern, as customers may be unaware of what sensitive information is being collected and how it is being used. It is important to keep customers informed and transparent of data policy and their ability to opt out. Even without using demographic variables, other information such as zip code and browsing behavior (i.e. more minorities utilizing Google in comparison to the other Channels) can also act as proxies for race and income discrimination and result in biased outcomes. Ethical responsibilities extend beyond compliance, instead companies should ensure that the data collected is only what's necessary, conduct regular fairness audits, and maintain a transparent data policy.

APPENDIX

Question 2: Did the coupon increase transactions? Did it increase revenues?

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the AB test data from the Excel file
df_ab = pd.read_excel("Artea Data.xlsx", sheet_name="AB_test")
df_nx = pd.read_excel("Artea Data.xlsx", sheet_name="Next_Campaign")

# Split the dataset into two groups based on test_coupon
# - Treatment group: received the coupon (test_coupon == 1)
# - Control group: did NOT receive the coupon (test_coupon == 0)
treatment = df_ab[df_ab["test_coupon"] == 1].copy()
control = df_ab[df_ab["test_coupon"] == 0].copy()

print("Treatment (coupon) users:", len(treatment))
print("Control (no coupon) users:", len(control))
```

Treatment (coupon) users: 2502
Control (no coupon) users: 2498

```
In [2]: # Average number of transactions made by users who received the coupon
avg_trans_treatment = treatment["trans_after"].mean()

# Average number of transactions made by users who did NOT receive the coupon
avg_trans_control = control["trans_after"].mean()

print("Average transactions after experiment (Treatment - coupon):", avg_trans_trea
print("Average transactions after experiment (Control - no coupon):", avg_trans_con

# Incremental change in transactions due to the coupon
transaction_lift = avg_trans_treatment - avg_trans_control

print("Incremental change in transactions due to coupon:", transaction_lift)
```

Average transactions after experiment (Treatment - coupon): 0.1518784972022382
Average transactions after experiment (Control - no coupon): 0.1257005604483587
Incremental change in transactions due to coupon: 0.0261779367538795

```
In [3]: # Group by the coupon test column and sum the transactions
transaction_compare = df_ab.groupby('test_coupon')['trans_after'].sum()

# Plot
ax = transaction_compare.plot(kind='bar', color=['lightcoral', 'lightgreen'], figsi

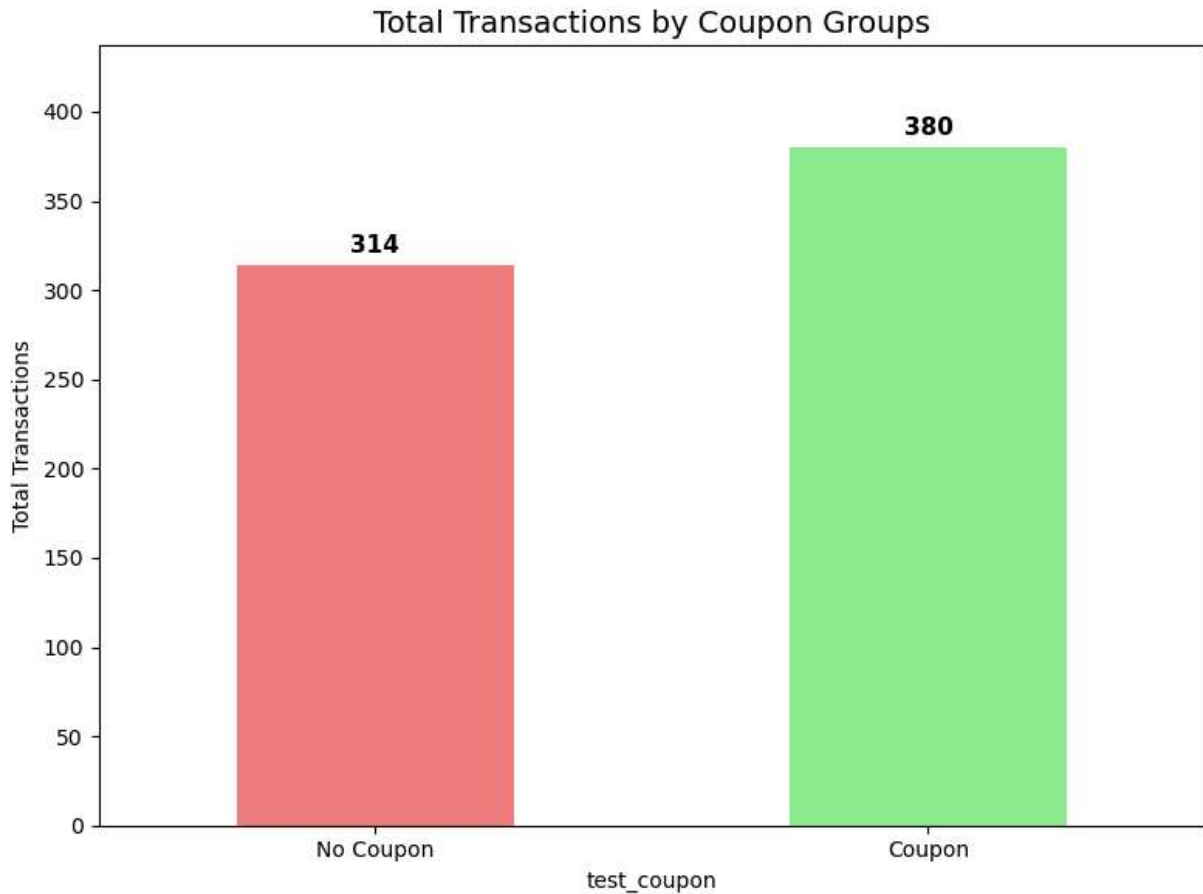
# Add the values on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height():}',
```

```

        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center', va='center',
        xytext=(0, 9),
        textcoords='offset points',
        fontsize=11, fontweight='bold')

plt.title('Total Transactions by Coupon Groups', fontsize=14)
plt.ylabel('Total Transactions')
plt.xticks([0, 1], ['No Coupon', 'Coupon'], rotation=0)
plt.ylim(0, transaction_compare.max() * 1.15) # Adds a little extra headroom for the
plt.tight_layout()
plt.show()

```



```

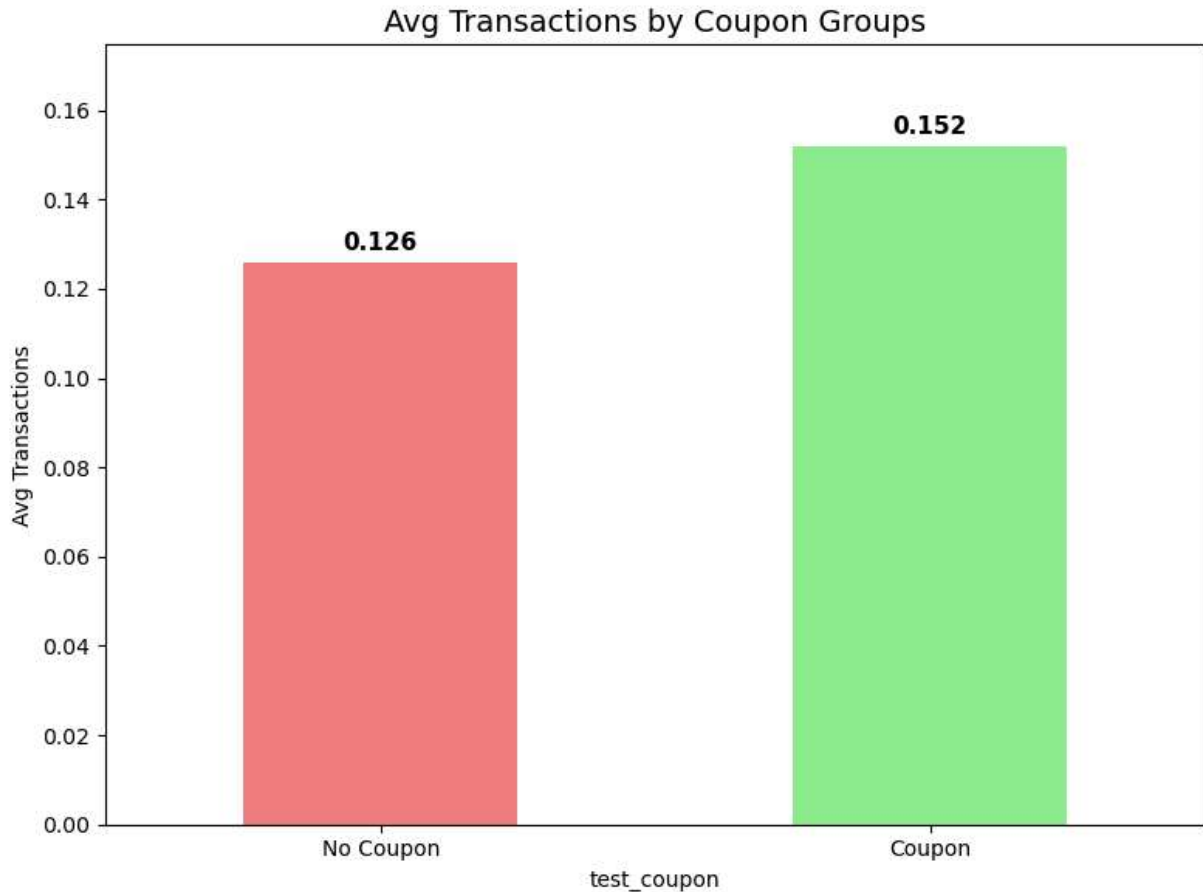
In [4]: # Average number of transactions after the experiment for each group
transaction_avg_compare = df_ab.groupby('test_coupon')['trans_after'].mean()

# Create the plot and assign it to an 'ax' (axis) variable
ax = transaction_avg_compare.plot(kind='bar', color=['lightcoral', 'lightgreen'], f

# Add the values on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height():.3f}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 9),
                textcoords='offset points',
                fontsize=11, fontweight='bold')

```

```
# Standard formatting
plt.title('Avg Transactions by Coupon Groups', fontsize=14)
plt.ylabel('Avg Transactions')
plt.xticks([0, 1], ['No Coupon', 'Coupon'], rotation=0)
plt.ylim(0, transaction_avg_compare.max() * 1.15) # Adds a little extra headroom fo
plt.tight_layout()
plt.show()
```



```
In [5]: # Average revenue generated by users who received the coupon
avg_rev_treatment = treatment["revenue_after"].mean()

# Average revenue generated by users who did NOT receive the coupon
avg_rev_control = control["revenue_after"].mean()

print("Average revenue after experiment (Treatment - coupon):", avg_rev_treatment)
print("Average revenue after experiment (Control - no coupon):", avg_rev_control)

# Incremental change in revenue due to the coupon
revenue_lift = avg_rev_treatment - avg_rev_control
print("Incremental change in revenue due to coupon:", revenue_lift)
```

Average revenue after experiment (Treatment - coupon): 7.53867306155076
 Average revenue after experiment (Control - no coupon): 7.780168134507607
 Incremental change in revenue due to coupon: -0.24149507295684636

```
In [6]: # Average revenue
avg_rev = df_ab.groupby('test_coupon')['revenue_after'].mean()

# Create the plot and assign it to an 'ax' (axis) variable
```

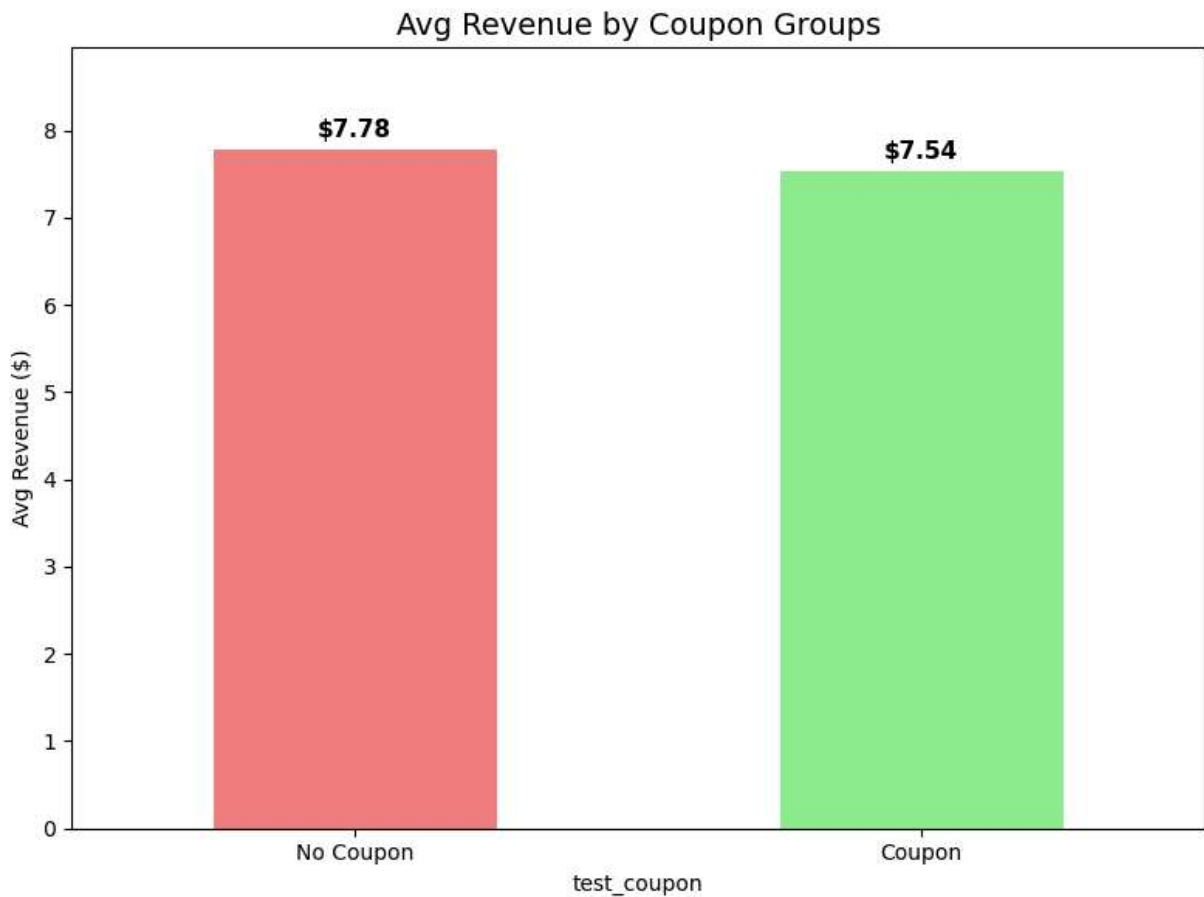
```

ax = avg_rev.plot(kind='bar', color=['lightcoral', 'lightgreen'], figsize=(8, 6))

# Add the values on top of the bars
for p in ax.patches:
    ax.annotate(f'${p.get_height():.2f}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 9),
                textcoords='offset points',
                fontsize=11, fontweight='bold')

# Standard formatting
plt.title('Avg Revenue by Coupon Groups', fontsize=14)
plt.ylabel('Avg Revenue ($)')
plt.xticks([0, 1], ['No Coupon', 'Coupon'], rotation=0)
plt.ylim(0, avg_rev.max() * 1.15) # Adds a little extra headroom for the labels
plt.tight_layout()
plt.show()

```



```

In [7]: # Group by the coupon test column and sum the revenue
rev_compare = df_ab.groupby('test_coupon')['revenue_after'].sum()

# Create the plot and assign it to an 'ax' (axis) variable
ax = rev_compare.plot(kind='bar', color=['lightcoral', 'lightgreen'], figsize=(8, 6))

# Add the values on top of the bars
for p in ax.patches:
    ax.annotate(f'${p.get_height():.2f}',

```

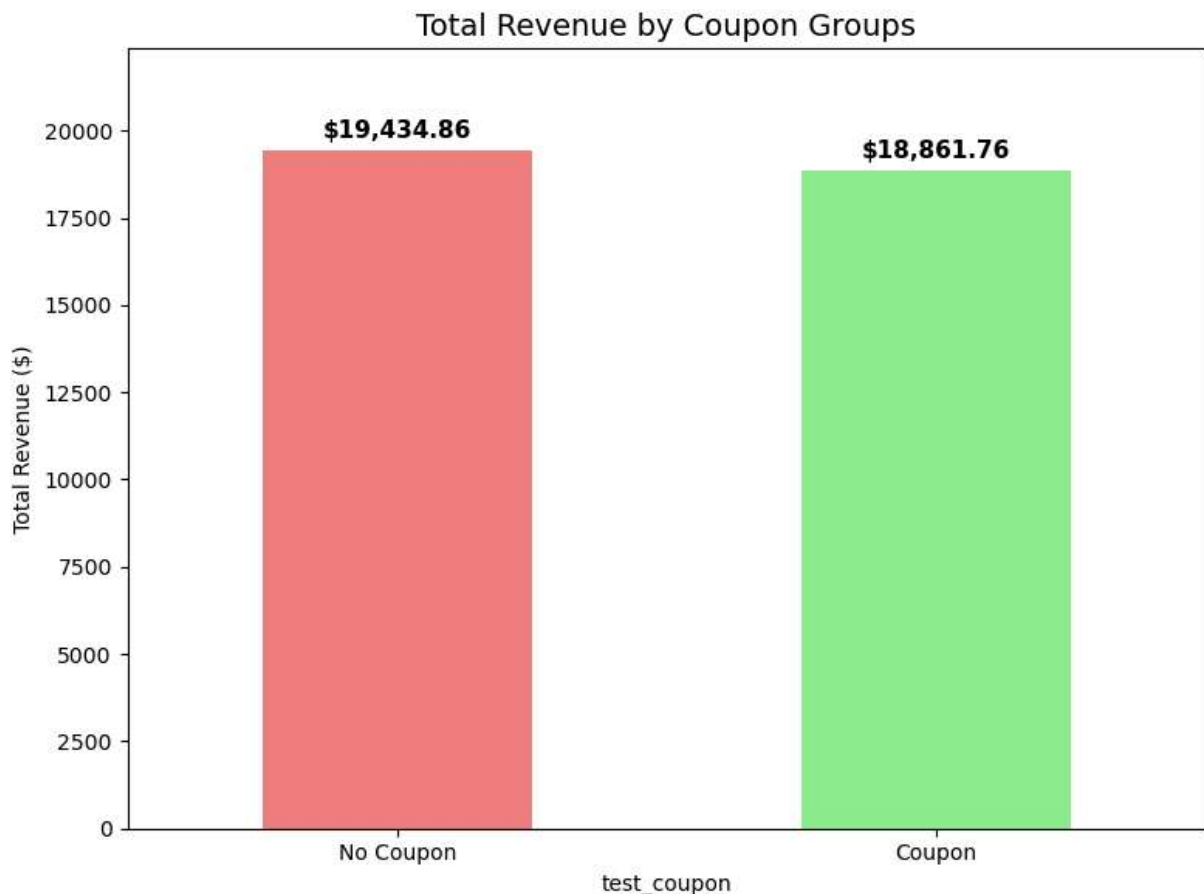
```

        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center', va='center',
        xytext=(0, 9),
        textcoords='offset points',
        fontsize=11, fontweight='bold')

# Standard formatting

plt.title('Total Revenue by Coupon Groups', fontsize=14)
plt.ylabel('Total Revenue ($)')
plt.xticks([0, 1], ['No Coupon', 'Coupon'], rotation=0)
plt.ylim(0, rev_compare.max() * 1.15) # Adds a little extra headroom for the labels
plt.tight_layout()
plt.show()

```



In [8]: `from scipy import stats`

```

# T-test for transactions
t_stat_trans, p_val_trans = stats.ttest_ind(treatment["trans_after"], control["tran
print("Transaction Significance:")
print(f"T-statistic: {t_stat_trans:.4f}")
print(f"P-value: {p_val_trans:.4f}")
print("Significant?" , "Yes" if p_val_trans < 0.05 else "No")

# T-test for revenue
t_stat_rev, p_val_rev = stats.ttest_ind(treatment["revenue_after"], control["revenu
print("\nRevenue Significance:")
print(f"T-statistic: {t_stat_rev:.4f}")

```



```
print(f"P-value: {p_val_rev:.4f}")
print("Significant?", "Yes" if p_val_rev < 0.05 else "No")
```

Transaction Significance:

T-statistic: 2.2072

P-value: 0.0273

Significant? Yes

Revenue Significance:

T-statistic: -0.3613

P-value: 0.7179

Significant? No

- The coupon did increase transactions — the average number of transactions for coupon recipients was 0.1519 compared to 0.1257 for the control group, representing a 20.8% increase. This effect is statistically significant ($p=0.027$), confirming the coupon genuinely drove more purchases and the difference is not due to chance.
- The coupon did not meaningfully increase revenues. Coupon recipients generated an average of \$7.54 compared to \$7.78 for the control group, a decrease of \$0.24 per user. The p-value of 0.718 confirms this difference is not statistically significant. This makes intuitive sense: while the coupon nudged more people to buy, the 20% discount ate into margins and offset any revenue gains, resulting in no net revenue lift.

Question 3: Which customers should be targeted in the next targeting campaign? How many transactions and how much revenues would you expect from that targeting strategy? What are the risks of this targeting strategy?

Approach Hybrid (Regression-Informed Segmentation)

Steps:

1. Run regression on A/B data with interaction terms.
2. Identify attributes that significantly affect coupon response.
3. Use those attributes to build meaningful customer segments.
4. Compute uplift within each segment (treatment – control).
5. Rank segments and select those with positive uplift.
6. Apply targeting rule to Next_Campaign dataset.
7. Estimate expected transactions and revenue.

```
In [9]: import statsmodels.formula.api as smf
features = [
    'shopping_cart',      # cart activity
    'weeks_since_visit',  # recency
    'browsing_minutes',   # browsing engagement
    'num_past_purch',     # purchase history
    'spent_last_purchase', # last purchase value
    'channel_acq'         # acquisition channel
```

```

]

df_ab_reg = df_ab[features + ['test_coupon', 'trans_after', 'revenue_after']].dropna()

print("Rows used for regression:", len(df_ab_reg))
df_ab_reg.head()

```

Rows used for regression: 5000

Out[9]:

	shopping_cart	weeks_since_visit	browsing_minutes	num_past_purch	spent_last_purchase
0	0	6	1	6	62.99
1	1	0	7	2	53.99
2	0	3	4	3	88.99
3	0	1	19	1	68.99
4	0	4	20	3	66.49



In [10]: *# Regression to identify attributes that influence coupon effectiveness*
Interaction terms (test_coupon:variable) tell us how the coupon effect
changes depending on customer characteristics.

```

rev_model = smf.ols(
    'revenue_after ~ test_coupon + shopping_cart + weeks_since_visit + '
    'browsing_minutes + num_past_purch + spent_last_purchase + C(channel_acq) + '
    'test_coupon:shopping_cart + test_coupon:weeks_since_visit + '
    'test_coupon:browsing_minutes + test_coupon:num_past_purch + '
    'test_coupon:spent_last_purchase + test_coupon:C(channel_acq)',
    data=df_ab_reg
).fit()

# Print regression summary
print(rev_model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          revenue_after    R-squared:                0.177
Model:                  OLS              Adj. R-squared:           0.174
Method:                 Least Squares    F-statistic:             56.35
Date:                   Thu, 12 Mar 2026  Prob (F-statistic):       6.52e-194
Time:                   19:13:02         Log-Likelihood:          -22419.
No. Observations:      5000             AIC:                    4.488e+04
Df Residuals:          4980             BIC:                    4.501e+04
Df Model:               19
Covariance Type:       nonrobust
=====

```

```

=====
                                coef    std err          t      P>|t|      [0.
025      0.975]
-----
Intercept                -2.4944      1.391      -1.794      0.073      -5.
221      0.232
C(channel_acq)[T.2]        5.2308      1.160       4.508      0.000       2.
956      7.506
C(channel_acq)[T.3]        4.4410      1.023       4.341      0.000       2.
435      6.447
C(channel_acq)[T.4]        5.2979      2.133       2.484      0.013       1.
117      9.479
C(channel_acq)[T.5]        9.0388      2.900       3.117      0.002       3.
354     14.723
test_coupon              -1.0287      1.973      -0.521      0.602      -4.
897      2.839
test_coupon:C(channel_acq)[T.2]  1.6582      1.631       1.017      0.309      -1.
539      4.855
test_coupon:C(channel_acq)[T.3]  3.1126      1.457       2.137      0.033       0.
257      5.968
test_coupon:C(channel_acq)[T.4]  2.6748      2.950       0.907      0.365      -3.
109      8.459
test_coupon:C(channel_acq)[T.5] -0.4824      4.102      -0.118      0.906      -8.
524      7.560
shopping_cart             7.6928      0.948       8.116      0.000       5.
835      9.551
weeks_since_visit        -1.1200      0.190      -5.895      0.000      -1.
492     -0.748
browsing_minutes          0.1511      0.063       2.403      0.016       0.
028      0.274
num_past_purch            3.8584      0.192     20.091      0.000       3.
482      4.235
spent_last_purchase       -0.0221      0.009      -2.509      0.012      -0.
039     -0.005
test_coupon:shopping_cart    3.1323      1.354       2.313      0.021       0.
478      5.787
test_coupon:weeks_since_visit  0.0026      0.269       0.010      0.992      -0.
525      0.531
test_coupon:browsing_minutes  0.0170      0.089       0.190      0.849      -0.
158      0.192
test_coupon:num_past_purch   -1.2772      0.262      -4.868      0.000      -1.
792     -0.763
test_coupon:spent_last_purchase 0.0135      0.012       1.120      0.263      -0.

```

```

010      0.037
=====
Omnibus:                3269.938    Durbin-Watson:                1.956
Prob(Omnibus):           0.000    Jarque-Bera (JB):            46407.375
Skew:                    2.958    Prob(JB):                     0.00
Kurtosis:                16.703    Cond. No.                     1.45e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.45e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Three interactions are significant.

- Customers who added items to their shopping cart respond more positively to the coupon ($p=0.021$), suggesting they need a small nudge to convert.
- Instagram-acquired customers also respond better ($p=0.033$) in comparison to baseline (channel 1), indicating this segment is more promotion-responsive.
- Customers with more past purchases respond worse ($p=0.000$), meaning loyal customers would likely have bought anyway, so giving them a discount simply erodes margin.

```

In [11]: # Create customer segments using variables that affect coupon response

df_ab_seg = df_ab.copy()

# Cart intent segment
df_ab_seg["Cart_Intent"] = df_ab_seg["shopping_cart"].map({
    1: "Added_to_Cart",
    0: "No_Cart"
})

# Past purchase history segments
df_ab_seg["Past_Purchase_Group"] = pd.cut(
    df_ab_seg["num_past_purch"],
    bins=[-0.001, 0, 2, 5, float("inf")],
    labels=["0 Purchases", "1-2 Purchases", "3-5 Purchases", "6+ Purchases"]
)

# Acquisition channel segment
df_ab_seg["Acquisition_Channel"] = df_ab_seg["channel_acq"].astype(str)

```

```

In [12]: # Compute uplift (treatment - control) for each segment

segment_cols = [
    "Cart_Intent",
    "Past_Purchase_Group",
    "Acquisition_Channel"
]

def uplift_by_segment(df, seg_col):

```

```

# Average outcomes by segment and coupon group
m = (df.groupby([seg_col, "test_coupon"])[["trans_after", "revenue_after"]]
      .mean()
      .unstack("test_coupon"))

# Ensure order: control(0), treatment(1)
m = m.reindex(columns=[0, 1], level=1)

m.columns = [
    "trans_control",
    "trans_treat",
    "rev_control",
    "rev_treat"
]

# Calculate uplift
m["transaction_uplift"] = m["trans_treat"] - m["trans_control"]
m["revenue_uplift"] = m["rev_treat"] - m["rev_control"]

# Segment size
m["segment_size"] = df.groupby(seg_col).size()

return m.reset_index()

# Compute uplift tables for all segmentation variables
uplift_tables = {col: uplift_by_segment(df_ab_seg, col) for col in segment_cols}

# Results
for name, table in uplift_tables.items():
    print(f"\nUplift by {name}")
    print(table.round(3))

```

Uplift by Cart_Intent

	Cart_Intent	trans_control	trans_treat	rev_control	rev_treat \
0	Added_to_Cart	0.215	0.292	13.147	14.528
1	No_Cart	0.087	0.096	5.482	4.742

	transaction_uplift	revenue_uplift	segment_size
0	0.077	1.381	1464
1	0.008	-0.740	3536

Uplift by Past_Purchase_Group

	Past_Purchase_Group	trans_control	trans_treat	rev_control	rev_treat \
0	0 Purchases	0.052	0.073	3.156	3.685
1	1-2 Purchases	0.074	0.116	4.469	5.749
2	3-5 Purchases	0.147	0.169	9.316	8.112
3	6+ Purchases	0.550	0.532	34.198	26.811

	transaction_uplift	revenue_uplift	segment_size
0	0.020	0.529	1665
1	0.043	1.280	1830
2	0.022	-1.204	1039
3	-0.018	-7.387	466

Uplift by Acquisition_Channel

	Acquisition_Channel	trans_control	trans_treat	rev_control	rev_treat \
0	1	0.079	0.059	4.922	2.804
1	2	0.159	0.221	9.815	10.895
2	3	0.146	0.205	9.172	10.337
3	4	0.186	0.240	11.180	11.518
4	5	0.259	0.241	14.859	12.757

	transaction_uplift	revenue_uplift	segment_size
0	-0.020	-2.118	2035
1	0.061	1.080	1059
2	0.058	1.166	1552
3	0.054	0.339	238
4	-0.017	-2.103	116

C:\Users\sruth\AppData\Local\Temp\ipykernel_33052\963617024.py:11: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
m = (df.groupby([seg_col, "test_coupon"]))[["trans_after", "revenue_after"]]
```

C:\Users\sruth\AppData\Local\Temp\ipykernel_33052\963617024.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
m["segment_size"] = df.groupby(seg_col).size()
```

```
In [13]: # Identify segments with positive transaction AND revenue uplift
target_segments = {}
for name, table in uplift_tables.items():
    targets = table[
        (table["transaction_uplift"] > 0) &
        (table["revenue_uplift"] > 0)
    ].copy()
    targets = targets.sort_values("revenue_uplift", ascending=False)
    target_segments[name] = targets
```

```
print(f"\nTarget segments for {name}")
print(targets.round(3))
```

Target segments for Cart_Intent

	Cart_Intent	trans_control	trans_treat	rev_control	rev_treat \
0	Added_to_Cart	0.215	0.292	13.147	14.528

	transaction_uplift	revenue_uplift	segment_size
0	0.077	1.381	1464

Target segments for Past_Purchase_Group

	Past_Purchase_Group	trans_control	trans_treat	rev_control	rev_treat \
1	1-2 Purchases	0.074	0.116	4.469	5.749
0	0 Purchases	0.052	0.073	3.156	3.685

	transaction_uplift	revenue_uplift	segment_size
1	0.043	1.280	1830
0	0.020	0.529	1665

Target segments for Acquisition_Channel

	Acquisition_Channel	trans_control	trans_treat	rev_control	rev_treat \
2	3	0.146	0.205	9.172	10.337
1	2	0.159	0.221	9.815	10.895
3	4	0.186	0.240	11.180	11.518

	transaction_uplift	revenue_uplift	segment_size
2	0.058	1.166	1552
1	0.061	1.080	1059
3	0.054	0.339	238

Target customers who satisfy all three conditions:

1. Added items to the shopping cart
2. Have 0 to 2 past purchases
3. Were acquired via Facebook, Instagram, or Referral

```
In [14]: # Apply targeting rule to the Next_Campaign dataset
df_nx_seg = df_nx.copy()
# Create same segments as AB data
df_nx_seg["Cart_Intent"] = df_nx_seg["shopping_cart"].map({
    1: "Added_to_Cart",
    0: "No_Cart"
})

df_nx_seg["Past_Purchase_Group"] = pd.cut(
    df_nx_seg["num_past_purch"],
    bins=[-0.001, 0, 2, 5, float("inf")],
    labels=["0 Purchases", "1-2 Purchases", "3-5 Purchases", "6+ Purchases"]
)

df_nx_seg["Acquisition_Channel"] = df_nx_seg["channel_acq"].astype(str)

# Apply targeting rule
target_customers = df_nx_seg[
```

```

(df_nx_seg["Cart_Intent"] == "Added_to_Cart") &
(df_nx_seg["Past_Purchase_Group"].isin(["0 Purchases", "1-2 Purchases"])) &
(df_nx_seg["Acquisition_Channel"].isin(["2","3","4"]))
].copy()

print("Total customers in Next Campaign:", len(df_nx_seg))
print("Customers to target:", len(target_customers))
print("Targeting rate:", round(len(target_customers)/len(df_nx_seg)*100,2), "%")

```

Total customers in Next Campaign: 6000
 Customers to target: 668
 Targeting rate: 11.13 %

```

In [15]: # Estimate expected incremental transactions and revenue
target_size = len(target_customers)

# Uplift from AB test (Cart Intent segment)
transaction_uplift = 0.077
revenue_uplift = 1.381

expected_transactions = target_size * transaction_uplift
expected_revenue = target_size * revenue_uplift

print("Expected incremental impact of targeting strategy:")
print("Customers targeted:", target_size)
print("Expected incremental transactions:", round(expected_transactions,2))
print("Expected incremental revenue ($):", round(expected_revenue,2))

# Baseline revenue (control average for Added_to_Cart segment)
baseline_revenue_per_customer = 13.147
total_revenue = target_size * baseline_revenue_per_customer + expected_revenue
print("Expected total revenue ($):", round(total_revenue, 2))

```

Expected incremental impact of targeting strategy:
 Customers targeted: 668
 Expected incremental transactions: 51.44
 Expected incremental revenue (\$): 922.51
 Expected total revenue (\$): 9704.7

```

In [16]: # Combine the positive target segments into one dataframe for visualization
plot_df = pd.concat([
    target_segments["Cart_Intent"].assign(variable="Cart_Intent", segment=lambda x:
    target_segments["Past_Purchase_Group"].assign(variable="Past_Purchase_Group", s
    target_segments["Acquisition_Channel"].assign(variable="Acquisition_Channel", s
])

plot_df = plot_df.sort_values(by="revenue_uplift", ascending=False)
plt.figure(figsize=(10,6))
plt.bar(
    plot_df["segment"].astype(str) + " (" + plot_df["variable"] + ")",
    plot_df["revenue_uplift"],
    color="green"
)

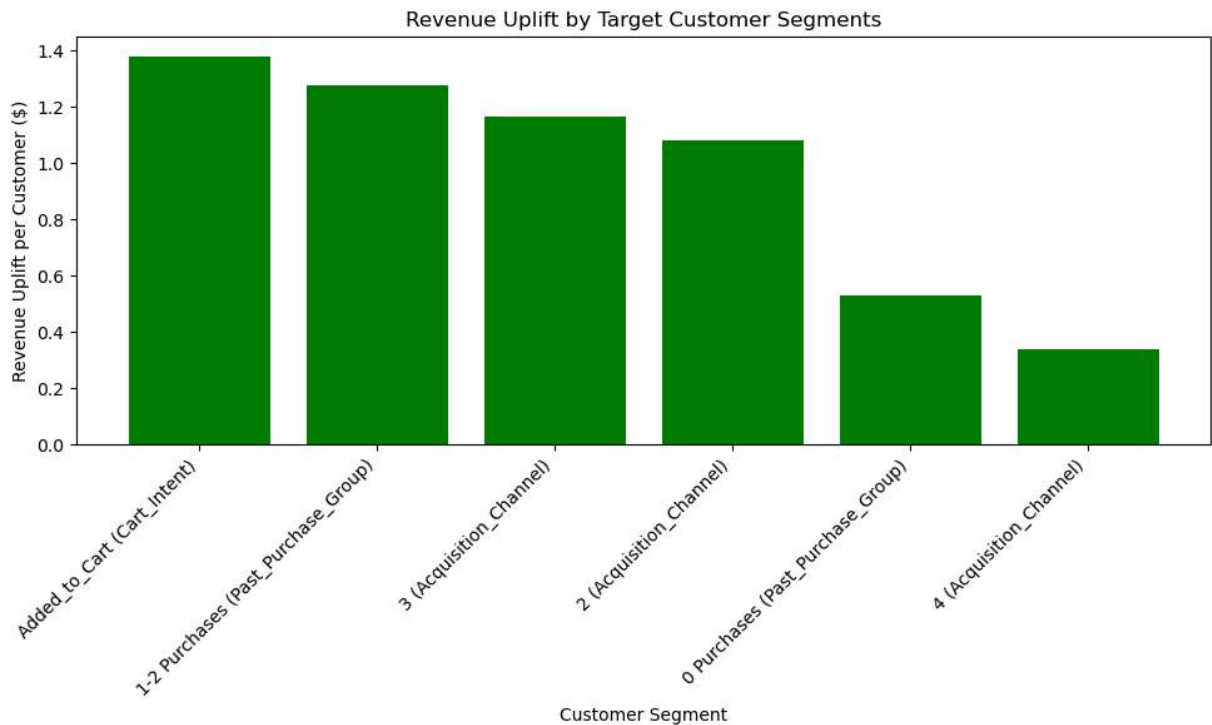
plt.xticks(rotation=45, ha="right")
plt.ylabel("Revenue Uplift per Customer ($)")
plt.xlabel("Customer Segment")

```



```
plt.title("Revenue Uplift by Target Customer Segments")

plt.tight_layout()
plt.show()
```



4. What do you learn from the demographics data? Would you change change the targeting policy from the first part of the case?

```
In [17]: import pandas as pd
import matplotlib.pyplot as plt

df_b = pd.read_excel("Artea (B) Data.xlsx", sheet_name="AB_test_demog")
df_nx_b = pd.read_excel("Artea (B) Data.xlsx", sheet_name="Next_Campaign_demog")

# Distribution of demographic variables
print("=== Dataset Overview ===")
print("Total customers:", len(df_b))
print("\nMinority distribution:")
print(df_b['minority'].value_counts())
print(f"Minority %: {df_b['minority'].mean()*100:.1f}%")

print("\nNon-male distribution:")
print(df_b['non_male'].value_counts())
print(f"Non-male %: {df_b['non_male'].mean()*100:.1f}%")

# Average transactions and revenue by demographic group
print("\n=== Avg Transactions by Minority x Coupon ===")
print(df_b.groupby(['minority', 'test_coupon'])['trans_after'].mean().round(3))

print("\n=== Avg Revenue by Minority x Coupon ===")
print(df_b.groupby(['minority', 'test_coupon'])['revenue_after'].mean().round(3))
```

```

print("\n=== Avg Transactions by Non_Male x Coupon ===")
print(df_b.groupby(['non_male', 'test_coupon'])['trans_after'].mean().round(3))

print("\n=== Avg Revenue by Non_Male x Coupon ===")
print(df_b.groupby(['non_male', 'test_coupon'])['revenue_after'].mean().round(3))

# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Revenue by minority
rev_minority = df_b.groupby(['minority', 'test_coupon'])['revenue_after'].mean().unstack
rev_minority.plot(kind='bar', ax=axes[0,0], color=['lightcoral', 'lightgreen'])
axes[0,0].set_title('Avg Revenue by Minority Status')
axes[0,0].set_xlabel('')
axes[0,0].set_xticklabels(['Non-Minority', 'Minority'], rotation=0)
axes[0,0].legend(['No Coupon', 'Coupon'])
axes[0,0].set_ylabel('Avg Revenue ($)')

# Transactions by minority
trans_minority = df_b.groupby(['minority', 'test_coupon'])['trans_after'].mean().unstack
trans_minority.plot(kind='bar', ax=axes[0,1], color=['lightcoral', 'lightgreen'])
axes[0,1].set_title('Avg Transactions by Minority Status')
axes[0,1].set_xlabel('')
axes[0,1].set_xticklabels(['Non-Minority', 'Minority'], rotation=0)
axes[0,1].legend(['No Coupon', 'Coupon'])
axes[0,1].set_ylabel('Avg Transactions')

# Revenue by non_male
rev_nonmale = df_b.groupby(['non_male', 'test_coupon'])['revenue_after'].mean().unstack
rev_nonmale.plot(kind='bar', ax=axes[1,0], color=['lightcoral', 'lightgreen'])
axes[1,0].set_title('Avg Revenue by Gender')
axes[1,0].set_xlabel('')
axes[1,0].set_xticklabels(['Male', 'Non-Male'], rotation=0)
axes[1,0].legend(['No Coupon', 'Coupon'])
axes[1,0].set_ylabel('Avg Revenue ($)')

# Transactions by non_male
trans_nonmale = df_b.groupby(['non_male', 'test_coupon'])['trans_after'].mean().unstack
trans_nonmale.plot(kind='bar', ax=axes[1,1], color=['lightcoral', 'lightgreen'])
axes[1,1].set_title('Avg Transactions by Gender')
axes[1,1].set_xlabel('')
axes[1,1].set_xticklabels(['Male', 'Non-Male'], rotation=0)
axes[1,1].legend(['No Coupon', 'Coupon'])
axes[1,1].set_ylabel('Avg Transactions')

plt.tight_layout()
plt.show()

```

=== Dataset Overview ===

Total customers: 5000

Minority distribution:

minority

0 3958

1 1042

Name: count, dtype: int64

Minority %: 20.8%

Non-male distribution:

non_male

1 2900

0 2100

Name: count, dtype: int64

Non-male %: 58.0%

=== Avg Transactions by Minority x Coupon ===

minority test_coupon

0 0 0.136

1 0.173

1 0 0.085

1 0.070

Name: trans_after, dtype: float64

=== Avg Revenue by Minority x Coupon ===

minority test_coupon

0 0 8.425

1 8.616

1 0 5.367

1 3.384

Name: revenue_after, dtype: float64

=== Avg Transactions by Non_Male x Coupon ===

non_male test_coupon

0 0 0.117

1 0.134

1 0 0.132

1 0.165

Name: trans_after, dtype: float64

=== Avg Revenue by Non_Male x Coupon ===

non_male test_coupon

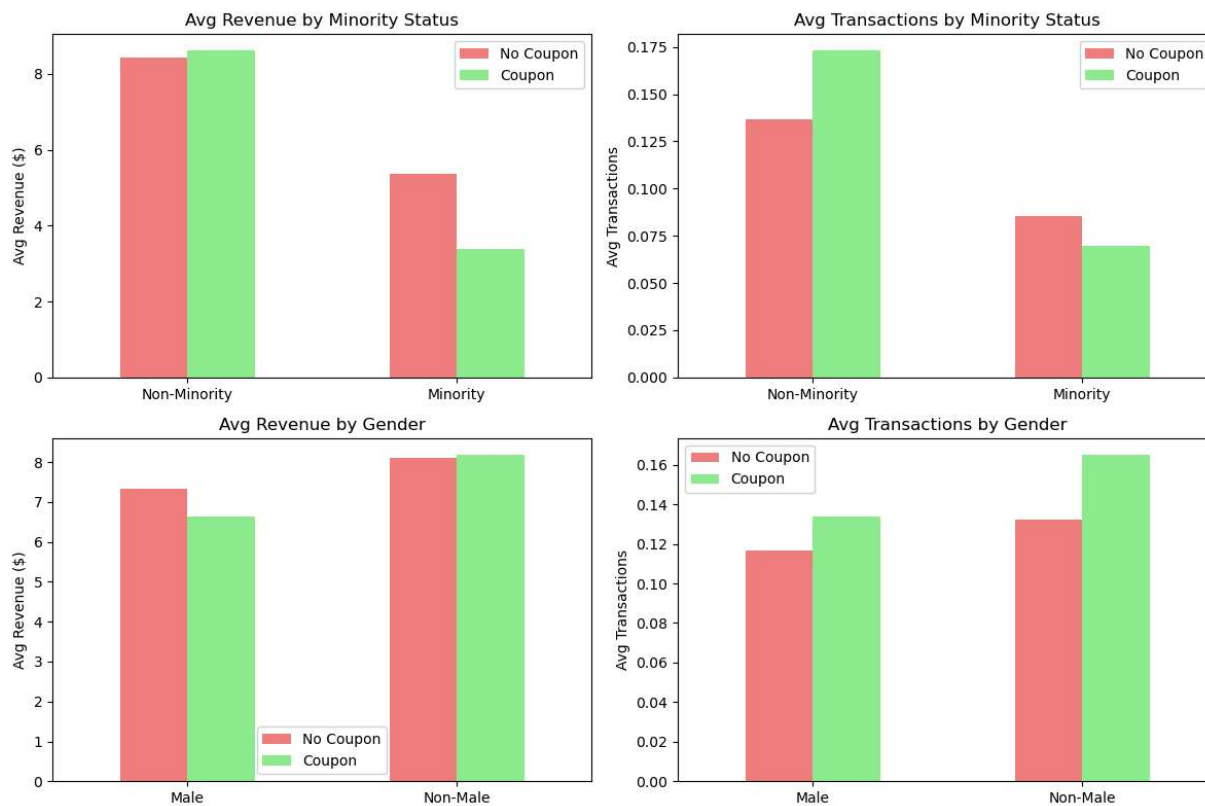
0 0 7.334

1 6.641

1 0 8.101

1 8.193

Name: revenue_after, dtype: float64



```
In [18]: import statsmodels.formula.api as smf

# Regression with demographic interaction terms
rev_model_b = smf.ols(
    'revenue_after ~ test_coupon + minority + non_male + '
    'shopping_cart + weeks_since_visit + browsing_minutes + '
    'num_past_purch + spent_last_purchase + C(channel_acq) + '
    'test_coupon:minority + test_coupon:non_male + '
    'test_coupon:shopping_cart + test_coupon:weeks_since_visit + '
    'test_coupon:browsing_minutes + test_coupon:num_past_purch + '
    'test_coupon:spent_last_purchase + test_coupon:C(channel_acq)',
    data=df_b
).fit()

print(rev_model_b.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          revenue_after    R-squared:                0.179
Model:                  OLS              Adj. R-squared:          0.175
Method:                 Least Squares    F-statistic:             47.09
Date:                   Thu, 12 Mar 2026  Prob (F-statistic):       9.36e-193
Time:                   19:13:34         Log-Likelihood:          -22414.
No. Observations:       5000            AIC:                    4.488e+04
Df Residuals:           4976            BIC:                    4.503e+04
Df Model:               23
Covariance Type:        nonrobust
=====

```

```

=====
                                coef    std err          t      P>|t|      [0.
025      0.975]
-----
Intercept                -2.4403      1.487      -1.641      0.101      -5.
356      0.475
C(channel_acq)[T.2]         5.1926      1.237       4.199      0.000       2.
768      7.617
C(channel_acq)[T.3]         4.2769      1.112       3.844      0.000       2.
096      6.458
C(channel_acq)[T.4]         5.3291      2.172       2.454      0.014       1.
071      9.587
C(channel_acq)[T.5]         9.0711      2.919       3.107      0.002       3.
348     14.794
test_coupon              -0.6956      2.103      -0.331      0.741      -4.
819      3.428
test_coupon:C(channel_acq)[T.2] 1.4435      1.737       0.831      0.406      -1.
963      4.850
test_coupon:C(channel_acq)[T.3] 2.6002      1.585       1.640      0.101      -0.
508      5.708
test_coupon:C(channel_acq)[T.4] 2.4182      3.007       0.804      0.421      -3.
477      8.313
test_coupon:C(channel_acq)[T.5] -0.5450      4.134      -0.132      0.895      -8.
650      7.559
minority                 -0.0768      1.170      -0.066      0.948      -2.
370      2.216
non_male                  1.3139      1.180       1.113      0.266      -1.
000      3.628
shopping_cart             7.8719      0.961       8.193      0.000       5.
988      9.755
weeks_since_visit         -1.1195      0.190     -5.891      0.000      -1.
492     -0.747
browsing_minutes          0.0927      0.082       1.132      0.258      -0.
068      0.253
num_past_purch            3.8595      0.192     20.102      0.000       3.
483      4.236
spent_last_purchase        -0.0222      0.009     -2.514      0.012      -0.
039     -0.005
test_coupon:minority       -0.7810      1.663     -0.470      0.639      -4.
040      2.478
test_coupon:non_male       2.3524      1.689       1.392      0.164      -0.
960      5.664
test_coupon:shopping_cart   3.4710      1.375       2.525      0.012       0.

```

```

776      6.166
test_coupon:weeks_since_visit      0.0057      0.269      0.021      0.983      -0.
522      0.534
test_coupon:browsing_minutes      -0.0878      0.117      -0.752      0.452      -0.
316      0.141
test_coupon:num_past_purch      -1.2797      0.262      -4.878      0.000      -1.
794      -0.765
test_coupon:spent_last_purchase      0.0139      0.012      1.146      0.252      -0.
010      0.038
=====
Omnibus:      3267.777      Durbin-Watson:      1.954
Prob(Omnibus):      0.000      Jarque-Bera (JB):      46479.112
Skew:      2.954      Prob(JB):      0.00
Kurtosis:      16.719      Cond. No.      1.48e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.48e+03. This might indicate that there are strong multicollinearity or other numerical problems.

The demographic interaction terms (test_coupon:minority and test_coupon:non_male) are both insignificant ($p=0.639$ and $p=0.164$ respectively), meaning the coupon effect does not statistically differ by minority status or gender. Although exploratory analysis showed minority customers had lower revenue in the treatment group, the regression confirms this pattern is not statistically robust. Therefore, the targeting policy from Part A does not need to change — shopping cart activity and past purchases remain the key targeting variables.

```
In [20]: pd.crosstab(df_b["channel_acq"], df_b["minority"])
```

```

Out[20]:      minority      0      1
channel_acq
1      1180    855
2       998     61
3      1453     99
4       223     15
5       104     12

```

```

In [23]: # Map channel numbers to names
channel_map = {
    1: "Google",
    2: "Facebook",
    3: "Instagram",
    4: "Referral",
    5: "Other"
}

```

```

df_b["channel_name"] = df_b["channel_acq"].map(channel_map)

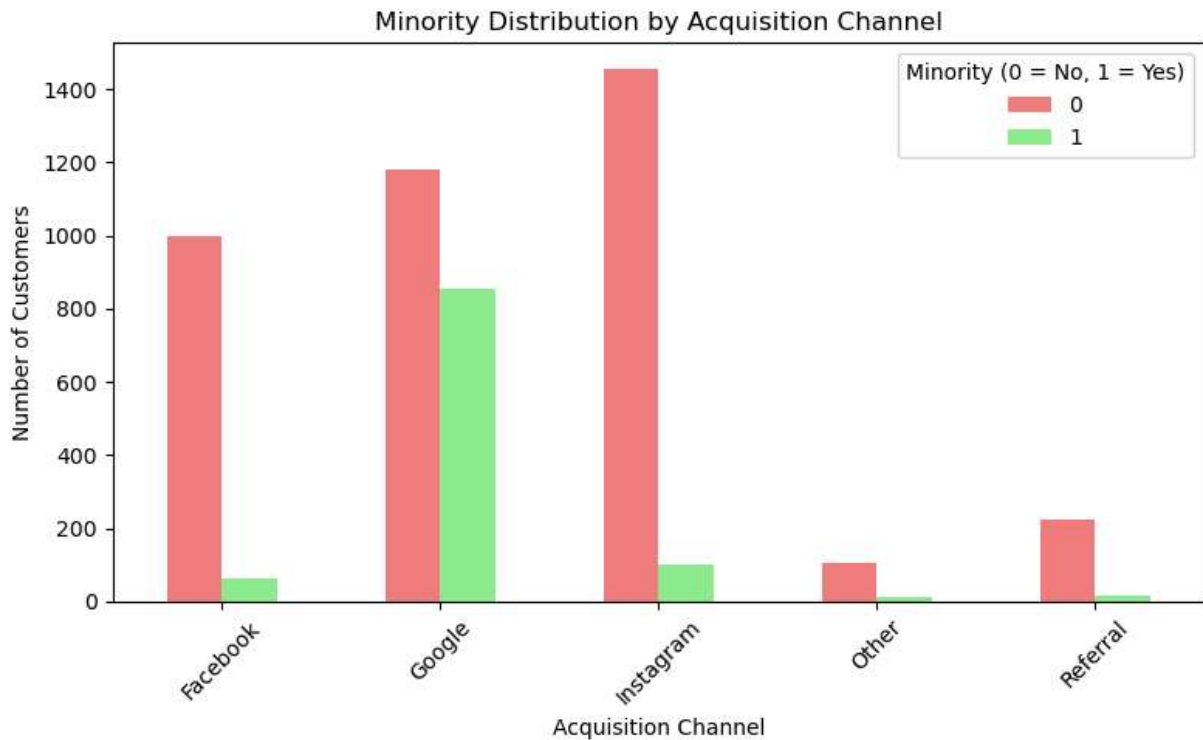
# Count minorities by acquisition channel
counts = pd.crosstab(df_b["channel_name"], df_b["minority"])

# Plot
counts.plot(
    kind="bar",
    figsize=(8,5),
    color=['lightcoral', 'lightgreen']
)

plt.xlabel("Acquisition Channel")
plt.ylabel("Number of Customers")
plt.title("Minority Distribution by Acquisition Channel")
plt.xticks(rotation=45)
plt.legend(title="Minority (0 = No, 1 = Yes)")

plt.tight_layout()
plt.show()

```



```

In [26]: import matplotlib.pyplot as plt

# Extract p-values for the demographic interaction terms
pvals = rev_model_b.pvalues[[
    "test_coupon:minority",
    "test_coupon:non_male"
]]

# Convert to dataframe
pvals_df = pvals.reset_index()
pvals_df.columns = ["Variable", "p_value"]

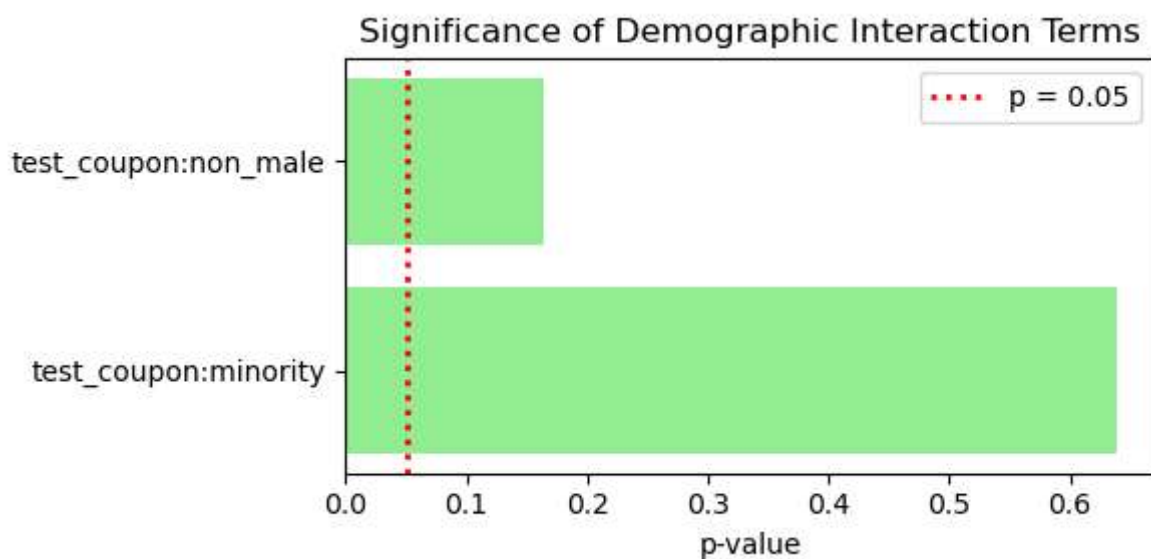
```

```
# Plot
plt.figure(figsize=(6,3))

plt.barh(
    pvals_df["Variable"],
    pvals_df["p_value"],
    color="lightgreen"
)

# Red dotted significance threshold
plt.axvline(x=0.05, color="red", linestyle="dotted", linewidth=2, label="p = 0.05")

plt.xlabel("p-value")
plt.title("Significance of Demographic Interaction Terms")
plt.legend()
plt.tight_layout()
plt.show()
```



In []: